

Conversion de données

Java est très strict du point de vue des données et il est parfois intéressant de *convertir* une donnée d'un type primitif vers un autre type primitif. Il faut néanmoins faire attention lors de conversions car il y a risque de perdre de l'information. En effet, imaginons vouloir convertir un `int` qui vaut 1254 en un `byte`. Ça ne sera pas possible puisque qu'avec le type `byte`, on ne peut représenter que des valeurs comprises entre -127 et 128. Si on fait la conversion, on va donc perdre de l'information.

Type de conversion

Il y a deux types de conversion. Les conversions sans perte d'information et celles qui impliquent une perte d'information. Il n'y a pas de perte d'information lorsque la grandeur du nombre converti est conservée.

Conversion sans perte d'information

Il y a 19 *conversions sans perte d'informations*, reprises sur la figure 18. Ces conversions ne perdent jamais d'information du point de vue de la quantité, mais les conversions vers un type primitif décimal peuvent mener à une *perte de précision*. Par exemple, lorsqu'on convertit un `int` ou un `long` vers un `double` ou un `float`, les chiffres les moins significatifs peuvent être perdus, la grandeur du nombre restant la même.

De	Vers
byte	short, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

Conversions sans perte d'information.

Conversion avec perte d'information

Il y a 22 *conversions avec perte d'information*. Il faut bien entendu essayer de les éviter puisqu'elles peuvent résulter en une perte d'information et de précision. Ces pertes ne se produisent pas dans tous les cas. Elles sont toutes reprises sur la figure 20. Ces pertes d'informations sont dues au fait qu'on passe d'un type de donnée qui utilise n bits vers un autre qui en utilise m , avec m étant strictement plus petit que n .

De	Vers
short	byte, char
char	byte, short
int	byte, short, char
long	byte, short, char, int
float	byte, short, char, int, long
double	byte, short, char, int, long, float

Conversions avec perte d'information.

Conversion implicite

Maintenant qu'on a vu les deux types de conversions qui existent, voyons quand celles-ci se produisent dans un programme Java. Il y a tout d'abord les *conversions implicites* qui se produisent automatiquement de manière implicite.

Conversion par affectation

Une *conversion par affectation* intervient lorsqu'une valeur d'un certain type est affectée à une variable d'un autre type. Lorsqu'on écrit $x = y$, la valeur de y va être convertie dans le type de la variable x . La conversion se fait donc à partir du type d'une expression vers le type d'une variable. Seules les conversions sans perte d'information sont possibles; les autres, ainsi que la conversion de `byte` vers `char`, vont produire une erreur lors de la compilation.

Promotion arithmétique

La *promotion arithmétique* intervient automatiquement lorsque certains opérateurs mathématiques doivent modifier leurs opérandes pour pouvoir exécuter l'opération. Par exemple, lorsque vous tentez de diviser un `float` par un `int`, la valeur du dénominateur sera promue en un type `float` avant la division, afin que le résultat de la division soit un `float`. Les opérandes d'un opérateur sont donc convertis vers le type requis par l'opérateur. Pour rappel, les opérateurs arithmétiques existent en quatre versions : `double`, `float`, `long` et `int`.

Soient les instructions suivantes :

```
float num = 10.0F;
int denom = 2;
System.out.println (num / denom);
```

L'exécution du programme affiche `5.0` à l'écran puisqu'un des deux opérandes de l'opérateur `/` est de type `float`, ce qui implique que le résultat sera

également un `float`. La valeur de la variable `denom` est convertie en un `float` avant la division.

Quels sont les opérateurs qui vont convertir automatiquement leurs opérandes ? Il y a tout d'abord les opérateurs unaires `-`, `+`, `--` et `++`. Si l'opérande est de type `byte`, `short` ou `char`, il est automatiquement converti en un `int`. Dans tous les autres cas, il n'est pas converti et conserve son type.

Enfin, il reste les opérateurs `+`, `-`, `*`, `/`, `%`, `<`, `<=`, `>`, `>=`, `==` et `!=`. Pour ceux-ci, il faut regarder le type des deux opérandes, et appliquer les règles suivantes dans l'ordre :

1. Si l'un des opérandes est de type `double`, l'autre est converti en `double` ;
2. Si l'un des opérandes est de type `float`, l'autre est converti en `float` ;
3. Si l'un des opérandes est de type `long`, l'autre est converti en `long` ;
4. Dans tous les autres cas, les deux opérandes sont convertis en `int`.

Cast ou conversion explicite

Les conversions qu'on vient de voir se font de manière implicite, automatiquement. Mais on ne peut faire que des conversions sans perte d'information. Il est néanmoins parfois utile de réaliser des conversions avec perte d'information ou de convertir un `byte` en un `char`. Pour cela, on doit faire un **cast**, aussi appelé conversion explicite.

On fait un cast grâce à l'**opérateur de cast**. Il s'agit d'un opérateur unaire, de priorité 3, associatif de droite à gauche. Grâce à cet opérateur, on va pouvoir convertir une donnée dans un type spécifié, que la conversion soit avec ou sans perte d'information.

L'opérateur est constitué de deux parenthèses entre lesquelles se trouve un type de donnée, précisément celui vers lequel la conversion doit avoir lieu. Ainsi, pour convertir une donnée en `int`, la forme prise par l'opérateur est `(int)`.

L'exemple suivant montre plusieurs conversions utilisant l'opérateur de cast. Il y a une conversion de `int` vers `float` (sans perte d'information), de `int` vers `byte` (avec perte d'information) et enfin de `byte` vers `char`.

```
int x = 292;

float f = (float) x; // Conversion sans perte d'information
byte b = (byte) x;   // Conversion avec perte d'information
char c = (char) b;   // Conversion d'un byte vers un char

System.out.println (f);
System.out.println (b);
System.out.println (c);
System.out.println ((byte) c);
```

Voici ce qui est affiché à l'écran après exécution de ce programme :

```
292.0  
36  
$  
36
```